Links:

1. `<dl>`, `<dt>`, `<dd>` Tag (HTML Definition List)

These tags are used together to display term–description pairs like in glossaries, documentation, or FAQs.

Tag	Meaning
`<dl>`	Definition list wrapper
`<dt>`	Definition term
`<dd>`	Definition description

Example:

```HTML
<dl>
  <dt>Python</dt>
  <dd>A high-level programming language.</dd>

  <dt>Flask</dt>
  <dd>A lightweight web framework in Python.</dd>
</dl>
```

---

2. Macros in Jinja2

Macros in Jinja are like functions for HTML templates. They help **avoid repetition**.

Example:

HTML

```
{% macro input_field(name, label, type="text") %}
  <label>{{ label }}</label>
  <input type="{{ type }}" name="{{ name }}">
{% endmacro %}

<form>
    {{ input_field("username", "Username") }}
    {{ input_field("password", "Password", "password") }}
</form>
```

---

3. Template Inheritance in Jinja (Very Useful in Flask)

Jinja supports inheritance via `block` and `extends`. This is how we reuse a layout (`base.html`).

base.html

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}Welcome{% endblock %}</title>
</head>
<body>
    <header>My Website</header>
    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

child.html

```HTML
{% extends "base.html" %}
{% block title %}Home{% endblock %}
{% block content %}
  <p>Hello from Home Page!</p>
{% endblock %}
```

---

4. `set` in Jinja

Used to create variables inside templates.

```HTML
{% set name = "Himanshu" %}
<p>Hello {{ name }}</p>

{% set total = price * quantity %}
<p>Total cost: {{ total }}</p>
```

---

5. Jinja Filters

Filters transform data inside templates.

Filter	Purpose	Example	
`sort`	Sort list	`{{ nums	sort }}`
`reverse`	Reverse sequence	`{{ nums	reverse }}`
`groupby`	Group items by attribute	`{{ users	groupby('role') }}`
`upper/lower`	Case conversion	`{{ name	upper }}`
`join`	List to string	`{{ items	join(', ') }}`

Example:

```HTML
{% set numbers = [5, 3, 9, 1] %}
Sorted: {{ numbers|sort }}  ⟶ [1,3,5,9]
```

link: link

---

6. `curl`

Command	Description
`curl <URL>`	Basic GET request.
`curl -v <URL>`	Verbose output with headers.
`curl -L <URL>`	Follow redirects.
`curl -O <URL>`	Download a file with its original name.
`curl -H`	Add custom headers.
`curl -X POST`	Send a POST request.
`curl -X POST -d`	Send form data with a POST request.
`curl -X POST -H "Content-Type: application/json" -d`	Send JSON data.
`curl <URL> -o <filename>`	Save output to a file.
`curl -i <URL>`	Include headers in the response.
`curl -F`	Upload a file to a server.
`curl -w`	Display timing details of the request. (custom output format for information about the transfer, rather than the transferred data itself)
`curl -s -o /dev/null -w`	Get HTTP status code only.

7. URL Parameters in Flask

Flask routes can capture parameters from the URL using converters.

Converter	Example	Type
`<int:var>`	`/user/5`	Integer
`<string:var>`	`/user/john`	Default type
`<float:var>`	`/num/5.6`	Float
`<path:var>`	`/files/some/file.txt`	Path
`<uuid:var>`	`/order/uuid`	UUID

Example:

PYTHON

```python
@app.route('/user/<int:user_id>')
def user_profile(user_id):
    return f"User ID = {user_id}"
```

---

8. Query Parameters

These are passed after `?` in the URL.

```
/search?query=python&page=2
```

Example (Flask):

PYTHON

```python
@app.route('/search')
def search():
    q = request.args.get('query')
    page = request.args.get('page', default=1, type=int)
    return f"Searching for {q} on page {page}"
```

---

9. HTML `<input>` Tag

Used in forms.

```HTML
<input type="text" name="username">
<input type="password" name="password">
<input type="email" name="email">
<input type="submit" value="Submit">
<select>
    <option name="same">option1</option>
    <option name="same">option2</option>
</select>
```

---

10. Route With and Without Forward Slashes in Flask

```PYTHON
@app.route('/hello')
def hello():
    return "Hello"
```

– Requires exact `/hello`

PYTHON

```python
@app.route('/hello/')
def hello2():
    return "Hello"
```

– Accessible via `/hello/` **AND** `/hello`

> ✅ Best practice: use trailing slash for directory-like resources.

---

in-memory data structures

```python
names = {0: 'Yashvi', 1:'Baskaran', 2:'Himanshu'}
courses = {0: 'DataSci', 1:'Intro to EE circuits', 2:'Quantum Mechanics'}
rels = [(0,0),(1,2),(0,2),(2,1)]
class Student:
    idnext = 0
    def __init__(self, name):
        self.name = name
        self.id = Student.idnext
        Student.idnext +=1
```

1. Server crashes → on restart → load back from saved disk `csv/serialised pickle`
2. Data entry errors → less likely
3. ❌ Duplicates
4. Auto-initialize → ✅ unique but preferred is tell underlying DB as multiple users+load

[list2table]

- ## Spreadsheets
  - **Lookup & Cross-Referencing Sheets:** Harder to perform lookups across multiple sheets or files.
    Functions like `VLOOKUP`, `HLOOKUP`, and `INDEX/MATCH` exist but become complex with scale.
    No built-in referential integrity; manual effort required.
  - **Stored Procedures & Functionality:** Limited to basic formulas and scripting (e.g., Google Apps Script, VBA in Excel).
    Not designed for complex business logic or reusable procedures.
  - **Atomic Operations:** No clear definition or support for atomic (all-or-nothing) operations.
    Changes are cell-based and can be partially applied.
- ## RDBMS (SQL)
  - **Lookup & Cross-Referencing Sheets:** Powerful JOIN operations for cross-referencing tables.
    Foreign keys enforce data relationships and integrity.
  - **Stored Procedures & Functionality:** Supports robust stored procedures, triggers, and functions for encapsulating business logic.
    Can automate, validate, and process data efficiently within the database.

- **Atomic Operations:** Full support for atomic transactions (ACID properties).
  Ensures data consistency and rollback on failure.

NoSQL (MongoDB, CouchDB)

- **Lookup & Cross-Referencing Sheets:** Document-based; cross-referencing is possible but not as straightforward as SQL JOINs.
  Data often denormalized for performance.
- **Stored Procedures & Functionality:** Generally, no concept of stored procedures.
  Some systems (e.g., MongoDB) allow server-side scripts, but functionality is limited compared to SQL.
- **Atomic Operations:** Varies by system:
  MongoDB supports atomic operations at the document level.
  CouchDB provides atomicity at the document level; multi-document transactions are complex or unsupported.

Consider the following two tables in an SQLite database:

Table: `Gifts`

GiftID	GiftName	Price
1	Teddy Bear	250
2	Coffee Mug	150
3	Keychain	100
4	Notebook	180
5	Wall Clock	300

Table: `Reviews`

ReviewID	GiftID	Rating
1	1	4.2
2	2	4.5
3	3	3.9
4	4	4.7
5	5	4.0

Which of the following SQL queries correctly retrieves a list of **unique gift names** with a **price less than 200** rupees and a **rating above 4**? (**Note:** Displaying the rating in the result is optional, but duplicate gift names should not appear.)

```
SELECT GiftName, Price
FROM Gifts,Reviews
WHERE Price < 200 AND Rating > 4;
```

```
SELECT g.GiftName, g.Price
FROM Gifts g
JOIN Reviews r ON g.GiftID = r.GiftID
WHERE g.Price < 200 AND r.Rating > 4;
```

```
SELECT g.GiftName, g.Price, r.Rating
FROM Gifts, Reviews
WHERE g.Price < 200
AND r.Rating > 4;
```

```
SELECT g.GiftName, g.Price, r.Rating
FROM Gifts g, Reviews r
WHERE g.GiftID = r.GiftID
AND g.Price < 200
AND r.Rating > 4;
```

12. SQLAlchemy Queries (ORM Style)

Insert

PYTHON

```python
user = User(name="Alice", email="alice@email")
db.session.add(user)
db.session.commit()
```

Select

PYTHON

```python
users = User.query.all()
user = User.query.filter_by(name="Alice").first()
user = User.query.get(1)
```

Update

PYTHON

```python
user = User.query.get(1)
user.name = "Alicia"
db.session.commit()
```

Delete

```python
db.session.delete(user)
db.session.commit()
```